Generative AI and Large Language Models

BMED365: Topic List G01–G20

BMED365

Spring 2027

Overview

1. Introduction to Generative AI
2. Transformer Architecture
3. LLM Fundamentals
4. Prompt Engineering
5. Context Engineering
6. Challenges with LLM
7. Models and Advanced Concepts

G01: Define generative AI and distinguish from discriminative AI

Discriminative AI:

- Learns to **classify** or distinguish between categories
- Models $P(y|x)$ - probability of class y given input x
- Examples: "Is this cancer?", "Which disease?"
- CNN for image classification

Generative AI:

- Learns to **create** new content
- Models $P(x)$ or $P(x|y)$ - the data distribution itself
- Examples: Text, images, code, music
- LLMs, diffusion models

Key difference

Discriminative: Input $x \rightarrow$ Category/Label y | **Generative:** Prompt $\rightarrow$ New content

Medical application

Generative AI: Summarize medical records, generate reports, answer questions, synthesize training data

G02: Explain the self-attention mechanism at a conceptual level

Self-attention: "What is relevant for what?"

Intuition:

- For each word: Look at **all** other words in the sentence
- Compute a **weighted sum** based on relevance
- Captures dependencies over long distances

Example:

*"The patient took the **medication**, and **it** helped with the pain."*

- Self-attention connects "it" to "medication" (not "pain")

Query, Key, Value (QKV)

Each token generates three vectors:
Query "What am I looking for?" | **Key** "What can I offer?" | **Value** "What is my content?"

G03: Describe the transformer architecture (encoder-decoder)

Transformer Architecture (Vaswani et al., 2017):

Encoder:

- Processes input sequence
- Builds context-rich representation
- Used in: BERT, embeddings

Decoder:

- Generates output sequentially
- Uses masked self-attention
- Used in: GPT, Claude, Gemini

Full encoder-decoder:

- Translation, summarization
- T5, Flan-T5

Key components:

1. Multi-head self-attention
2. Feed-forward network
3. Layer normalization
4. Residual connections
5. Positional encoding

GPT = "Decoder-only"

Large language models (LLMs) typically use only the decoder part for text generation.

G04: Explain what tokens are and how tokenization works

Tokens = the "atoms" of text

- LLMs don't read letters or words - they read **tokens**
- Token ≈ subword, about 4 characters per token (for English)

Tokenization - example:

`"The patient has diabetes"` → `["The", " patient", " has", " diab", "etes"]`

BPE and language differences:

- Builds vocabulary iteratively
- GPT-4: approx. 100,000 tokens
- **Chinese:** Each character ≈ 1–2 tokens
- Simplified/traditional: Different tokens

Important to know:

- Long/rare words = more tokens
- Affects cost and speed
- `tiktoken` (OpenAI) for counting
- Different models = different tokenizers

Why tokens?

More efficient than letters (too short) or whole words (vocabulary too large).

G05: Understand the concept of context window

Context window = the model's "working memory"

- Maximum number of tokens the model can "see" at once
- Includes both **input** (prompt) and **output** (response)

Example context windows:

Model	Context	≈ Pages
GPT-4o	128K	∼150 pages
GPT-5	256K+	∼300 pages
Claude Opus 4.5	200K	∼250 pages
Gemini 3 Pro	2M+	Entire books!

Input vs. output:

- **Input context:** How much the model can read
- **Output context:** How long a response it can give
- Output often smaller (4K–32K tokens)

Limitations

Long texts must be truncated. The model "forgets" outside the window. Larger window = more expensive/slower.

G06: Explain what temperature means in text generation

Temperature = control over "creativity"

Technical:

- Scales logits before softmax: $p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$
- **Effect:** Low $T \to$ sharper distribution (highest logit dominates); High $T \to$ flatter distribution (more randomness)

Low temperature (0.0–0.3):

- More deterministic
- Chooses most likely tokens
- Consistent, "safe"
- ✔ Medical info, facts

High temperature (0.7–1.0+):

- More random/creative
- More surprising choices
- More variation
- ✔ Creative writing, brainstorming

Recommendation for medicine

Use **low temperature** (0.0–0.3) for fact-based tasks. Higher temperature increases hallucination risk.

G07: Apply zero-shot prompting

Zero-shot = no examples - ask the model to solve the task **without showing examples**

Examples of zero-shot prompts:

Zero-shot prompts

1. `Classify symptom as acute/chronic: "Headache for 3 months"`
2. `Summarize the medical note in 3 sentences.`
3. `Translate to patient-friendly language: "Bilateral pneumonia"`
4. `Extract all medications from this text.`
5. `Is this lab value abnormal? Hb 8.2 g/dL`

When does zero-shot work?

- The model has seen similar tasks
- The task is clearly defined

Limitations

For specialized tasks, zero-shot may give poor results. Then **few-shot** is needed.

G08: Apply few-shot prompting

Few-shot = some examples

- Give the model **2–5 examples** of desired input-output
- The model learns the pattern "in-context"

Few-shot prompt

```
Classify symptom duration:
Symptom: "Chest pain for 30 minutes" → Acute
Symptom: "Back pain for 2 years" → Chronic
Symptom: "Fever since yesterday" → Acute
--
Symptom: "Headache for 3 months" →
```

Tips for effective few-shot:

- Choose **representative** examples
- Show the **variation** in possible outputs
- Keep examples **consistent** in format
- Start with 3–5 examples, adjust as needed

G09: Apply Chain-of-Thought (CoT) prompting

Chain-of-Thought = show the reasoning

- Ask the model to **think step-by-step**
- Improves performance on complex reasoning tasks

Without CoT

```
Should the patient be referred?
→ "Yes"
```

With CoT

```
Think step-by-step:
1. Symptoms: X, Y, Z
2. Severity: Moderate
→ Conclusion: Should be referred
```

Variants:

- **Zero-shot CoT:** "Let's think step by step"
- **Few-shot CoT:** Show examples with reasoning

Built-in CoT in newer LLMs

ChatGPT o1/o3 (Thinking), Claude Opus 4.5 (Extended Thinking), Gemini 3 (Deep Think) have CoT built-in.

G10: Describe best practices for system prompts

System prompt = instruction that sets context

Elements of a good system prompt:

1. **Role:** "You are a medical assistant..."
2. **Context:** "The user is a healthcare professional..."
3. **Behavior:** "Answer concisely and precisely..."
4. **Limitations:** "Do not give diagnoses..."
5. **Format:** "Answer in English, use bullet points..."

Example system prompt

```
You are a medical AI assistant for healthcare professionals. Be precise and evidence-based. Reference sources when possible. Never give definitive diagnoses - suggest differential diagnoses. If uncertain, say so.
```

Tip

Test and iterate on system prompts! Small changes can have large effects.

G15: From Prompt Engineering to Context Engineering

Prompt Engineering vs. Context Engineering:

- **Prompt engineering:** Focus on crafting good questions
- **Context engineering:** Designing the *entire context window* strategically

Components of context engineering:

Component	Description	Medical example
System prompt	Role and instructions	"Clinical decision support"
User prompt	Specific question	"Evaluate the patient's symptoms"
Tools	Available actions	Lab lookup, ICD-10, drug database
RAG context	Retrieved information	Guidelines, records, articles
Conversation history	Previous dialog	Earlier questions and answers
Structured data	Formatted info	JSON with vital signs

Why important in medicine?

Rich context → safer, more relevant, and personalized responses

G16–G17: Model-specific prompting techniques

Three leading LLM providers with different strengths:

GPT-5.2 (OpenAI):

- Strongest on structured output
- Best tool integration
- Good instruction following

Well suited for:

- JSON extraction
- API integrations
- Agentic systems

Claude Opus 4.5:

- 200K token context
- Nuanced reasoning
- Constitutional AI

Well suited for:

- Ethical dilemmas
- Long record analysis
- Patient communication

Gemini 3 (Google):

- Multimodal (image+text)
- Real-time web search
- Scientific strength

Well suited for:

- Image analysis
- Literature search
- Calculations

Lab 3 Notebook 04

Contains detailed prompting guides for each model + comparison experiments

G18: Handle uncertainty and hallucination risk

Prompt patterns for medical uncertainty handling:

Anti-hallucination prompt (GPT-5.2 style)

```
<uncertainty_and_ambiguity>
- For ambiguous symptoms:  Present differential diagnoses with probability
- Clearly state knowledge limitations and update date
- Always refer to current guidelines and clinical judgment
- When uncertain:  Escalate to higher urgency level
</uncertainty_and_ambiguity>
```

Self-check for high-risk tasks:

1. Scan own response for unstated assumptions
2. Verify that numbers and claims are grounded in context
3. Avoid overly strong language (“always”, “guaranteed”)
4. Qualify conclusions with uncertainty level

Rule of thumb

AI = decision support, **not** decision maker. Always clinical validation!

G19–G20: Structured extraction and model comparison

G19: Structured extraction:

JSON schema for discharge summary

```
{
  "diagnosis": {
    "text": string,
    "icd10": string | null
  },
  "medications": [{
    "name": string,
    "dose": string
  }]
}
```

Important rules:

- Use `null` for missing fields
- Require source references
- Verify against original document

G20: Decision matrix:

Task	Model
Structured output	GPT-5.2
Ethical assessments	Claude
Image analysis	Gemini
Long context	Claude
Real-time search	Gemini
Tools/API	GPT-5.2

Practical tips:

1. Test same prompt on 2–3 models
2. Compare against evaluation criteria
3. Choose model based on task
4. Document choices for reproducibility

G11: Define hallucination and its implications

Hallucination = the model "makes up" incorrect information

- LLMs generate text that *sounds* correct but is *actually wrong*
- Also called *confabulation* - both terms are used in the literature

Types of hallucinations:

- **Factual errors:** Wrong statistics, dates, names
- **Fabricated sources:** References that don't exist
- **Logical errors:** Inconsistent reasoning

Development:

- Newer models hallucinate *less*
- Cannot be *completely* eliminated
- RAG and CoT reduce risk

Implications in medicine

Wrong dosage can be life-threatening. **Always verify** against reliable sources! LLM = decision support, **not** replacement for professional expertise.

G12: Know about GPT-5, Claude, Gemini and their applications

Model	Developer	Strengths	Application
GPT-5/o3	OpenAI	Multimodal, reasoning	ChatGPT, Copilot
Claude 4	Anthropic	200K context, "safe"	Document analysis
Gemini 3	Google	Integrated in Colab	Code help, search
Llama 4	Meta	Open source, local	Research

Local models:

- Ollama, LM Studio
- Distillation: Small model learns from large
- Quantization: Reduced precision (16→4 bit)
- GDPR-friendly, offline

chat.uib.no

Strengths: GDPR-ok, no data storage, free for UiB
Weaknesses: Older model, limited context, not multimodal

G13: Explain the concept of foundation model

Foundation model = pretrained on enormous amounts of data

- Trained on terabytes of text/images without specific task
- Can be adapted (*finetuned*) to many different tasks
- Costs millions to train from scratch

Characteristics:

1. **Large scale:** Billions of parameters
2. **Self-supervised learning:** No labels necessary
3. **Emergent capabilities:** Abilities that "emerge" at scale
4. **Adaptable:** Finetuning, prompting, RAG

Examples of foundation models (2025)

Text: GPT-5, Claude 4, Llama 4 | **Images:** DALL-E 3, Stable Diffusion 3 | **Multimodal:** Gemini 3, Sora

G14: Describe RAG (Retrieval-Augmented Generation)

RAG = connect LLM to external knowledge base

The problem RAG solves:

- LLM has "frozen" knowledge
- Can hallucinate facts
- Lacks domain-specific info

How RAG works:

1. **Query:** User asks question
2. **Retrieval:** Fetch documents
3. **Augmentation:** Add to prompt
4. **Generation:** LLM responds

Medical application

- Connect LLM to guidelines, drug databases, UpToDate
- Answer based on patient record (with permissions)
- Reduces hallucination by grounding answers in sources
- Clinical decision support with updated evidence
- Search institution-specific procedures and protocols

Summary: Generative AI and LLM

Fundamentals:

- G01: Generative vs. discriminative AI
- G02–G03: Self-attention and transformer architecture
- G04–G06: Tokens, context window, temperature

Prompt and context engineering:

- G07–G10: Zero-shot, few-shot, CoT, system prompts
- G15–G17: Context engineering, model-specific techniques
- G18–G20: Uncertainty handling, structured extraction

Challenges and advanced:

- G11: Hallucination - critical in a medical context
- G12–G14: Models (GPT-5.2, Claude, Gemini), RAG

Lab 3 Notebook 04

Detailed prompting guides for GPT-5.2, Claude Opus 4.5, and Gemini 3 with comparison experiments.